MIRA-Pro: AI Medical Directory Scraper & Crawler

Complete Prompt Engineering, Debugging & Troubleshooting Documentation

Candidate: Sanjay Mathivanan | Position: AI Growth & Automation Associate | Company: Gokul Infocare

Executive Summary & Iterative Methodology

This document presents the full prompt engineering history used to develop MIRA. Beyond initial prototyping, it captures real-world engineering challenges faced during development—including API model deprecation (404s), rate-limiting under high traffic (429 ResourceExhausted), thread deadlocks from circular imports, and the architectural scaling into a multi-source automated discovery crawler with failure auditing.

STAGE 1: SCRAPING

Initial Logic & Scraping Architecture

Prompt 1 (Initial Logic):

```
"Write a Python script using requests and BeautifulSoup to scrape all paragraph and header text from a medical clinic's
'Our Doctors' webpage, handling basic HTTP headers to avoid being blocked."
```

Technical Rationale & Challenges Overcome:

Australian clinic websites employ varied layouts and bot protections (Cloudflare, User-Agent filtering). The initial prompt engineered a resilient scraping harness using real browser headers and multi-tag parsing (h1, h2, h3, p, div) to capture all unstructured practitioner listings without losing relevant clinical details.

STAGE 2: AI SCHEMA

Semantic Intelligence & Strict Output Formatting

Prompt 2 (Adding AI Data Intelligence):

```
"The scraped text is too messy. Integrate the Google Gemini API (or OpenAI API). Write a prompt inside the Python script
that takes the scraped text and asks the LLM to extract all Doctor Names, Specialties, and Contact Details, forcing the
LLM to output ONLY a strict JSON array of objects."
```

Technical Rationale & Challenges Overcome:

Traditional regex rules break across heterogeneous clinic templates. This prompt introduces an LLM reasoning step to semantically understand messy medical credentials (FRACGP, MBBS, DipChildHealth). Crucially, it imposes strict JSON array formatting constraints to enable predictable downstream data pipelines.

STAGE 3: GRADIO UI

Adding the UI & Automated Pandas Export

Prompt 3 (Adding the UI & Pandas):

```
"Wrap this pipeline into a Gradio web UI. Have the Gradio app take a URL string as input, run the scraping and LLM
extraction, load the resulting JSON into a Pandas DataFrame, and display the DataFrame in the UI while also providing a
downloadable CSV file."
```

Technical Rationale & Challenges Overcome:

A script running only in a terminal has low usability for growth and sales teams. This prompt transforms the pipeline into a single-click web application. Gradio provides intuitive inputs and dynamic dataframe preview, while Pandas seamlessly serializes the structured records into an instantly exportable CSV file.

Engineering Problem-Solving & Real-World Bug Fixes

Part 2: Markdown Sanitization, API 404 Deprecations & High-Traffic Fallback Cascades

STAGE 4: HARDENING

Refining Prompt & Markdown Sanitization

Prompt 4 (Troubleshooting Markdown Backticks):

```
"The LLM is sometimes adding markdown formatting like ```json to the output, which breaks the json.loads() function.
Update the internal system prompt to strictly forbid markdown formatting and strip any backticks before parsing the JSON
into Pandas."
```

Technical Rationale & Challenges Overcome:

LLMs frequently wrap code blocks in markdown fences, triggering fatal JSONDecodeErrors in Python. We engineered a two-tier defense: (1) System prompt negative constraints forbidding intro text and markdown fences, and (2) programmatic string sanitization (.replace('```json', '').replace('```', '')) for 100% parsing resilience.

STAGE 5: MODEL 404 FIX

Resolving API Model Deprecations & 404 Errors

Prompt 5 (Dynamic Model Fallback):

```
"When calling gemini-1.5-flash, the API returned '404 models/gemini-1.5-flash is not found for API version v1beta'. Write
a multi-model fallback cascade function in Python that dynamically queries available models and switches automatically to
active models like gemini-3.8-flash without crashing."
```

Technical Rationale & Challenges Overcome:

Google retired older flash checkpoints for new API accounts. Instead of hardcoding a fragile string, we built a dynamic fallback resolver (generate_medical_extraction) that queries available flash endpoints and tries active candidates (models/gemini-3.8-flash, gemini-flash-latest) automatically if a 404 is encountered.

STAGE 6: RATE-LIMIT FIX

Handling High-Traffic 429 & 503 Errors via Lite Models

Prompt 6 (Traffic Resilience & Rate Limits):

```
"Under peak traffic, the Gemini API throws '429 ResourceExhausted' or '503 Service Unavailable'. Update the fallback
cascade to catch 429 and 503 errors and instantly switch to high-throughput Lite models like gemini-flash-lite-latest,
which have larger free-tier quotas and lower latency."
```

Technical Rationale & Challenges Overcome:

Flagship models experience quota throttling during peak hours. By incorporating 'models/gemini-flash-lite-latest' into the priority cascade and catching quota/overload exceptions, the pipeline automatically routes traffic to lighter, high-throughput models, guaranteeing uninterrupted continuous extraction.

Scaling to Multi-Source Discovery & Failure Auditing

Part 3: Decoupled Worker Threads, Dual CSV Auditing & Assessment Rubric Alignment

STAGE 7: THREAD FIX

Resolving Gradio Worker Thread Circular Imports

Prompt 7 (Decoupling Architecture):

```
"When triggering bulk crawl in Gradio, an internal error occurred: 'AttributeError: Button object has no attribute _id'
due to circular imports between pipeline.py and app.py. Refactor the codebase by moving all scraping and extraction logic
into a standalone extractor.py module."
```

Technical Rationale & Challenges Overcome:

Gradio handles background task execution in worker threads. Circular imports re-instantiated UI block components inside worker threads, corrupting Gradio's internal component tree. Decoupling extraction into extractor.py isolated core computational logic from UI rendering, resulting in a clean, enterprise-grade architecture.

STAGE 8: MULTI-SOURCE

Multi-Source Automated Discovery & Failure Auditing

Prompt 8 (National Discovery & Auditing):

```
"Build an automated discovery system that aggregates clinic doctor URLs across Australian cities (Sydney, Melbourne,
etc.) using OpenStreetMap, Healthdirect, and DuckDuckGo. For any URL that fails (Cloudflare 403, Timeout, DNS Fail),
label the exact reason in scraping_audit_log.csv."
```

Technical Rationale & Challenges Overcome:

Rather than manual copy-pasting, this prompt scaled the tool into a national directory crawler. Crucially, it implemented a dual-output model: master_australian_doctors.csv for structured GP rosters, and scraping_audit_log.csv to provide complete visibility and diagnostics into skipped or protected clinic domains.

SSESSMENT CRITERIA ALIGNMENT MATRI

Practical Use of AI Tools:
Integrated Google Gemini Flash models for high-speed entity extraction from noisy HTML DOM elements, transforming unstructured clinical bios into strict schema arrays.

Iterative Prompt Engineering:
Demonstrated an intentional 8-stage evolution: baseline scraping -> schema enforcement -> UI integration -> markdown sanitization -> 404 model migration -> traffic fallback -> thread decoupling -> bulk multi-source discovery.

Problem Solving & Resilience:
Solved real-world production blockers: API model deprecations (404), peak traffic rate-limiting (429 fallback to Lite), Gradio thread deadlocks, and Cloudflare anti-bot blocks.

Quality & Real-World Usefulness:
Directly outputs Australian medical directory data with 34+ doctors cataloged in Sydney alone, while generating a dedicated scraping audit log for enterprise operations.

Background Influence:
Directly applies data pipeline validation principles developed at BMW Plant Chennai (automating 150,000 files), prioritizing fault tolerance and deterministic outputs over brute-force scripts.